

Modelado matricial del embarque de un avión

TP1 — Aplicaciones Computacionales en Negocios

17 de septiembre de 2026

1. Idea general

Simulamos el embarque de un avión de **25 filas** y **4 asientos por fila** (100 pasajeros). Queremos medir cuánto tarda en llenarse según el *orden de embarque* (Random, Back-to-Front, WILMA, Steffen).

El modelo tiene dos piezas que conviene no confundir:

- **Fuente de verdad:** los objetos **Pasajero**. Cada pasajero guarda sus atributos fijos y su *historia* segundo a segundo.
- **Vista derivada:** la **matriz** del avión. Es una *foto* del estado físico en el instante actual, recalculada en cada paso de tiempo.

La matriz **no** reemplaza a la clase **Pasajero**: la complementa. Sirve para las preguntas *espaciales* (¿quién bloquea a quién?, ¿cuántos hay a bordo?), que en una lista de objetos serían incómodas de responder.

2. Por qué no alcanza una sola matriz

Una matriz de identificadores solo guarda *posición*. Pero un pasajero carga más información, y buena parte no es propia de una celda sino del pasajero:

Dato	¿Cabe en una matriz de id?
Posición actual	sí
Estado (camina / guarda / se sienta / ...)	no (haría falta otra matriz)
Carry-on (0/1)	no — es constante del pasajero
Velocidad, asiento destino	no — ídem
Temporizador de la acción en curso	no
Historia por segundo	no — la matriz es solo <i>ahora</i>

Conclusión: atributos e historia viven en **Pasajero**; la geometría del instante vive en la **matriz** derivada. Al escribir la posición en un solo lugar, es imposible que las dos representaciones se contradigan.

3. La grilla

$$\text{matriz} \in \mathbb{Z}^{25 \times 5}, \quad \text{matriz}[\text{f}, \text{c}] = \begin{cases} 0 & \text{celda vacía} \\ \text{id} > 0 & \text{pasajero ubicado en esa celda} \end{cases}$$

Como cada **id** es único y aparece en **una sola** celda mientras está a bordo, la matriz dice al instante dónde está cada uno.

Ejes

Filas (0–24 en NumPy): las 25 filas físicas. El avión numera de 1 a 25, así que

`fila_matriz = fila_avion - 1.`

Columnas (0–4): los cuatro asientos más el pasillo central.

col 0	col 1	col 2	col 3	col 4
ventana izq.	pasillo izq.	PASILLO	pasillo der.	ventana der.
lado -2	lado -1	(se camina)	lado 1	lado 2

La **columna 2** es el pasillo central: el único lugar donde un pasajero *camina*. Los sentados ocupan las columnas 0, 1, 3, 4.

Mapa lado → columna

```
def lado_a_col(lado):  
    return {-2: 0, -1: 1, 1: 3, 2: 4}[lado]
```

4. Ejemplo de foto

En el instante t , el pasajero 7 ya está sentado en la fila 2, lado -1 (col 1), y el pasajero 12 camina por el pasillo a la altura de la fila 2 (col 2):

	c0	c1	c2	c3	c4
f0	0	0	0	0	0
f1	0	7	12	0	0
f2	0	0	0	0	0
			⋮		

5. La matriz como vista derivada

La verdad es `pasajero.ubicacion[-1]` (la posición actual de cada uno). En cada paso de tiempo se **borra** la matriz y se **reconstruye** leyendo a los pasajeros:

```
def actualizar_matriz(self):  
    self.matriz[:] = 0                                # borro la foto anterior  
    for p in self.pasajeros:  
        fila, col = p.ubicacion[-1]                  # donde esta ahora  
        if fila >= 0:                                  # -1 = fuera del avion, no se dibuja  
            self.matriz[fila, col] = p.id
```

6. Para qué sirve la matriz

Detectar bloqueos

El pasillo (col 2) admite un solo pasajero por celda. Para avanzar de la fila f a la $f+1$:

```
if self.matriz[fila + 1, 2] == 0:    # pasillo libre adelante  
    # avanza  
else:  
    # bloqueado: espero (estado 2)
```

Contar a bordo

Sin recorrer la lista de pasajeros:

```
np.count_nonzero(self.matriz > 0)
```

Visualizar

Imprimir la matriz en cada paso da una animación del embarque.

7. El cuello de botella

Toda la demora del embarque nace en la columna 2. Un pasajero que guarda su carry-on en la fila 10 *tapa* a todos los que vienen atrás: ellos quedan en estado *bloqueado* hasta que el de adelante se siente y libere el pasillo. La representación matricial hace ese choque explícito y trivial de chequear: basta mirar la celda de adelante en la col 2.

8. Cómo encaja con la simulación

El orden de embarque (la salida de `orden_random`, `orden_steffen`, etc.) es la *cola*: define quién sube primero. El reloj avanza de a un segundo:

```
t = 0
while not avion.esta_lleno():
    for p in pasajeros:                # en el orden de la cola
        p.actualizar_estado(avion, t)  # appendea 1 valor a ubicacion y estado
    avion.actualizar_matriz()          # recalculo la foto
    t += 1
```

Como cada `actualizar_estado` agrega exactamente un valor por segundo, todas las listas `ubicacion` y `estado` terminan con largo *t*: queda la película completa del embarque, segundo a segundo, lista para medir tiempos o animar.